

CSS Teaching Notes

Topics: CSS Fundamentals · Positioning & Stacking · Pseudo-classes & Pseudo-elements

Part 1 — CSS Fundamentals

1.1 What CSS Does

HTML defines *structure* (what the content is). CSS defines *presentation* (how it looks). A CSS rule has three parts:

```
selector {  
  property: value; /* this line is a declaration */  
}  
  
/* Example */  
p {  
  color: black;  
  font-size: 16px;  
}
```

- **Selector** — which element(s) the rule targets.
- **Property** — the thing you are changing (color, font-size...).
- **Value** — what you are setting it to.
- **Declaration** — a single `property: value;` pair.
- **Declaration block** — everything inside the `{ }`.

1.2 Selectors

Element (type) selector

Targets every element of that tag name.

```
p      { color: black; } /* all <p> */  
h1     { font-size: 32px; } /* all <h1> */  
button { cursor: pointer; } /* all <button> */
```

Class selector (.)

Targets any element carrying that class. Reusable across many elements — this is the workhorse of CSS.

```
/* CSS */  
.btn      { padding: 8px 16px; }  
.text-big { font-size: 20px; }  
  
<!-- HTML -->  
<button class="btn">Save</button>  
<p class="text-big">Hello</p>  
  
<!-- an element can hold multiple classes -->  
<button class="btn text-big">Big Save</button>
```

ID selector (#)

Targets the single element with that id. An id must be **unique** on the page.

```
#main-header { height: 80px; }

<header id="main-header">...</header>
```

TEACHING POINT: Prefer classes for styling. IDs are hard to override (high specificity, see 1.4) and cannot repeat. Use IDs mainly for JS hooks or anchor links.

Grouping selector (,)

Apply the same declarations to several selectors at once. The comma means "OR".

```
h1, h2, h3 {
  font-family: Arial, sans-serif;
  margin: 0;
}
/* same as writing the block three times */
```

Descendant selector (space)

A space between two selectors targets elements nested *anywhere inside* an ancestor (child, grandchild, deeper).

```
/* any <a> inside .nav, at any depth */
.nav a { color: black; }

<nav class="nav">
  <ul>
    <li><a>Home</a></li> <!-- matched -->
  </ul>
</nav>
<a>Outside</a> <!-- NOT matched -->
```

Quick reference — related combinators (mention briefly)

Syntax	Name	Matches
A B	Descendant	B anywhere inside A
A > B	Child	B that is a <i>direct</i> child of A
A + B	Adjacent sibling	B immediately after A
A ~ B	General sibling	Any B after A, same parent
*	Universal	Every element

1.3 The Cascade

"Cascading" Style Sheets — when multiple rules touch the same element, the browser decides the winner in this order:

1. **Origin / importance** — author styles beat browser defaults; `!important` overrides normal rules (avoid it as a habit).
2. **Specificity** — the more specific selector wins (see 1.4).
3. **Source order** — if specificity ties, the rule written **last** wins.

```
p { color: black; }
p { color: gray; } /* wins: same specificity, comes later */
```

1.4 Specificity

Specificity is scored as three buckets (IDs, Classes, Elements). Compare left to right; a higher-left bucket always beats everything to its right.

Selector type	Weight (ID, Class, Element)	Example
Element / pseudo-element	0, 0, 1	<code>p, ::before</code>
Class / attribute / pseudo-class	0, 1, 0	<code>.btn, :hover</code>
ID	1, 0, 0	<code>#header</code>
Inline style	highest (beats above)	<code>style="..."</code>
<code>!important</code>	overrides all normal rules	—

Worked example

```
<p id="intro" class="lead">Hello</p>

p          { color: black; } /* 0,0,1 */
.lead      { color: blue;  } /* 0,1,0 <- beats element */
#intro     { color: green; } /* 1,0,0 <- beats class -> WINS */
```

Result: the text is green. Even 100 classes cannot outscore one ID.

RULE OF THUMB: Keep selectors as low-specificity as possible. When a style "won't apply," it is almost always losing a specificity or source-order battle — inspect in DevTools.

1.5 Colours

Four common ways to write a colour value:

Format	Example	Notes
Keyword	<code>color: black;</code>	~140 named colours
Hex	<code>#000000</code> / <code>#000</code>	RR GG BB, 00–FF each
RGB / RGBA	<code>rgb(0,0,0)</code> / <code>rgba(0,0,0,.5)</code>	A = alpha (opacity 0–1)
HSL / HSLA	<code>hsl(0,0%,0%)</code>	Hue, Saturation, Lightness

```
.box {
  color: #333;           /* text */
  background-color: #fff; /* background */
  border: 1px solid rgba(0,0,0,0.2);
}
```

1.6 Units

Unit	Type	Relative to	Use for
px	Absolute	Fixed device pixels	Borders, fine fixed sizes
%	Relative	The parent element	Fluid widths / layout
em	Relative	Font-size of the <i>current</i> element	Spacing that scales with local text
rem	Relative	Font-size of the root (<html>)	Consistent, predictable sizing

em vs rem — the key gotcha

em compounds when nested; rem always points to the root, so it does not compound.

```
html { font-size: 16px; } /* 1rem = 16px everywhere */

.parent { font-size: 20px; }
.parent .child {
  font-size: 2em; /* 2 x 20px = 40px (uses parent) */
  padding: 2rem; /* 2 x 16px = 32px (uses root) */
}
```

TEACHING POINT: Default to rem for font-size and layout spacing (predictable + accessible), px for borders, % for fluid widths, and em when you specifically want something to scale with its own text.

1.7 Fonts

```
body {
  font-family: "Helvetica Neue", Arial, sans-serif; /* font stack */
  font-size: 16px;
  font-weight: 400; /* 100-900; 400 normal, 700 bold */
  font-style: normal; /* or italic */
  line-height: 1.5; /* unitless = 1.5 x font-size */
}
```

- **Font stack** — list fallbacks left to right; the browser uses the first available. Always end with a generic family (serif, sans-serif, monospace).
- **line-height** — prefer a unitless number; it multiplies the element's font-size and inherits sensibly.

1.8 Text Properties

Property	Does	Common values
text-align	Horizontal alignment	left, right, center, justify
text-decoration	Lines on text	none, underline, line-through
text-transform	Letter casing	uppercase, lowercase, capitalize
letter-spacing	Space between letters	e.g. 0.5px, 0.05em
line-height	Space between lines	e.g. 1.5
font-weight	Boldness	normal, bold, 100–900

```
.heading {  
  text-align: center;  
  text-transform: uppercase;  
  letter-spacing: 1px;  
  text-decoration: none;  
}
```

MARGIN & PADDING: already covered — skipped as requested. Quick reminder only: `margin` = space *outside* the border; `padding` = space *inside* the border.

Session 1 — Positioning & Stacking

2.1 The `position` Property

`position` controls how an element is placed in the document, and whether the offset properties `top`, `right`, `bottom`, `left` do anything.

Value	Behaviour	Offsets work?	Removed from flow?
<code>static</code>	Default. Normal document flow.	No	No
<code>relative</code>	Stays in flow, then nudged by offsets from its own original spot.	Yes	No (original space kept)
<code>absolute</code>	Positioned relative to nearest <i>positioned</i> ancestor.	Yes	Yes
<code>fixed</code>	Positioned relative to the viewport; stays put on scroll.	Yes	Yes
<code>sticky</code>	Acts relative until a scroll threshold, then sticks like fixed.	Yes	No (until stuck)

static (the default)

```
.box { position: static; top: 50px; } /* top is IGNORED */
```

relative

Offsets move the element visually, but the space it originally occupied is preserved (other elements do not move in to fill it).

```
.box {  
  position: relative;  
  top: 10px;    /* pushes DOWN 10px from where it was */  
  left: 20px;   /* pushes RIGHT 20px */  
}
```

Two big uses: (1) small visual nudges, (2) making a box the **positioning context** for an `absolute` child.

absolute

Removed from normal flow. It is positioned against the nearest ancestor that has a position other than `static`. If none exists, it falls back to the page (`<html>`).

```
.card {  
  position: relative;    /* becomes the reference */  
}  
.badge {  
  position: absolute;  
  top: 8px;  
  right: 8px;           /* pin to top-right of .card */  
}
```

THE #1 PATTERN TO TEACH: parent `position: relative` + child `position: absolute`. Without a positioned parent, the absolute child escapes to the whole page.

fixed

Pinned to the viewport (browser window). Ignores scrolling — great for sticky navbars, "back to top" buttons, cookie banners, modals.

```
.navbar {
  position: fixed;
  top: 0; left: 0; right: 0; /* full-width bar glued to top */
  height: 60px;
}
```

sticky

Hybrid: behaves like `relative` while the element is on screen, then "sticks" at the given threshold as you scroll past it. Needs a threshold (e.g. `top: 0`) and enough scrollable parent height to have an effect.

```
.section-title {
  position: sticky;
  top: 0; /* sticks to the top edge once reached */
}
```

2.2 z-index & Stacking

When elements overlap, `z-index` decides who is on top. Higher number = closer to the viewer.

- `z-index` only applies to **positioned** elements (anything not `static`) and flex/grid children.
- Without `z-index`, later elements in the HTML paint on top of earlier ones.
- Values can be any integer (including negatives).

```
.modal { position: fixed; z-index: 1000; } /* on top */
.overlay { position: fixed; z-index: 999; } /* behind modal */
.content { position: relative; z-index: 1; }
```

GOTCHA — STACKING CONTEXTS: a `z-index: 9999` child cannot escape a parent that sits lower in its own stacking context. Certain properties (e.g. `position` + `z-index`, `opacity < 1`, `transform`) create a new stacking context, and children are only ranked *within* it. If `z-index` "isn't working," look at the parent.

2.3 overflow

Controls what happens when content is larger than its box.

Value	Effect
<code>visible</code>	Default. Content spills outside the box.
<code>hidden</code>	Overflowing content is clipped and hidden.
<code>scroll</code>	Always shows scrollbars (even if not needed).
<code>auto</code>	Scrollbars appear only when needed.

```
.panel {
  height: 200px;
  overflow: auto; /* scroll only when content is too tall */
}
.clip {
  overflow: hidden; /* also useful to trim/round content */
}
```

You can also control axes separately with `overflow-x` and `overflow-y`.

2.4 Pseudo-classes & Pseudo-elements

Both attach extra styling without adding HTML. Distinguish them by the punctuation:

- **Pseudo-class** — single colon `:` — styles an element in a particular **state**.
- **Pseudo-element** — double colon `::` — styles or creates a **part** of an element.

Pseudo-classes (state)

`:hover`

Applies while the pointer is over the element. Common for buttons and links.

```
.btn      { background: #eee; }  
.btn:hover { background: #ccc; cursor: pointer; }
```

`:focus`

Applies when an element is focused (clicked into, or tabbed to via keyboard). Vital for accessibility — never remove focus styling without a visible replacement.

```
input:focus {  
  outline: 2px solid #000;  
  outline-offset: 2px;  
}
```

Other useful state pseudo-classes (mention)

Selector	Applies when...
<code>:active</code>	Element is being pressed/clicked
<code>:visited</code>	A link has already been visited
<code>:first-child</code> / <code>:last-child</code>	Element is the first / last among siblings
<code>:nth-child(n)</code>	Matches by position, e.g. <code>:nth-child(even)</code>
<code>:disabled</code> / <code>:checked</code>	Form control state

Pseudo-elements (parts)

`::before` and `::after`

Insert generated content *inside* an element, before or after its real content. They **require** the `content` property to appear (even if empty: `content: ""`). The generated box is inline by default.

```

/* Add a small label before every .price */
.price::before {
  content: "$";
}

/* Decorative element with empty content */
.card::after {
  content: "";
  display: block;
  height: 2px;
  background: #000;
}

/* Add an arrow after external links */
.external::after {
  content: " \2192"; /* -> arrow */
}

```

TEACHING POINT: `::before` and `::after` are ordered relative to the element's text content: `::before` comes first, then the real content, then `::after`. They are purely decorative — do not put meaningful content there (screen readers may skip it).

Other pseudo-elements (mention)

<code>::first-line</code>	Styles just the first line of a block
<code>::first-letter</code>	Styles the first letter (drop-cap effect)
<code>::placeholder</code>	Styles input placeholder text
<code>::selection</code>	Styles highlighted/selected text

Combining them — a realistic example

```

/* underline that grows from the left on hover */
.link {
  position: relative;
  text-decoration: none;
}
.link::after {
  content: "";
  position: absolute;
  left: 0; bottom: -2px;
  width: 0;
  height: 2px;
  background: #000;
}
.link:hover::after {
  width: 100%; /* state (:hover) drives the part (::after) */
}

```

Recap Checklist

- Selectors: element, class, id, grouping (`()`), descendant (space) — plus specificity scoring (ID > class > element) and source order.
- Colours (hex / rgb / hsl), units (`px`, `%`, `em`, `rem` — em compounds, rem does not), fonts and text properties.

- Positioning: static, relative, absolute, fixed, sticky — and the relative-parent / absolute-child pattern.
- `z-index` needs positioning; watch out for stacking contexts. `overflow`: visible / hidden / scroll / auto.
- Pseudo-classes (`:hover` , `:focus`) = state; pseudo-elements (`::before` , `::after`) = parts, and need `content`.